

A Dataset Description

All experiments are conducted using data generated from a simulator P^* trained on data from the Norwegian university admissions database [22]. The use of a simulator is *necessary* due to the interactive nature of the problem, as evaluation cannot be performed on static datasets where outcomes are only observed for admitted students. This approach allows us to model realistic admission dynamics while maintaining full control over the experimental environment. It also allows for consistent and repeatable evaluation of different policies without requiring access to sensitive or real-time student data.

The original dataset is not publicly available, as it was obtained under a data-sharing agreement for research purposes. It includes academic features such as high school GPA, science scores, language scores, and additional admission-related attributes. Admission decisions are binary (0 = rejected, 1 = admitted), with an approximate acceptance rate of 70%.

To enhance realism, we train a Conditional Tabular Generative Adversarial Network (CTGAN) [41] for 100 epochs and generate 1,000 synthetic applicants. This ensures that the generated data remains statistically consistent with the original distribution. From these applicants, we identify the admitted students and simulate three course-specific GPA outcomes based on their academic profiles. These GPAs are clipped to lie within the range $[0, 4]$.

We preprocess the data by applying one-hot encoding to categorical variables and normalizing numerical features using Min-Max scaling. The generated course GPAs are also normalized to ensure consistency across models. Finally, the dataset is split into 70% training and 30% testing subsets for robust evaluation.

Experimental Setup. All experiments were conducted on a MacBook Pro equipped with an Apple M2 chip and 8 GB of RAM, running macOS Ventura. The implementation relies on Python, using PyTorch (v2.0), NumPy (v1.24), scikit-learn (v1.2), and the CTGAN model from the SDV library (v0.7.0). External GPU acceleration was not used. The experimental setup remains lightweight and fully reproducible on standard personal hardware.

Real-world admission cycles at large institutions can involve tens of thousands of applicants. The main computational bottlenecks in our framework are (i) CTGAN retraining at each stage, and (ii) the Monte Carlo estimation of the policy gradient, whose cost scales with the number of population samples n_x , action samples n_a , and population size N . We expect the policy optimization itself to scale reasonably (batched gradient computation is standard in PyTorch), but CTGAN retraining at every admission cycle could become a bottleneck at larger N or with more frequent stage updates; Investigating scalability on institutional-scale datasets constitutes an interesting direction for future work.

B Algorithms

In this section, we describe the practical implementation of the proposed policy optimization framework. We consider two settings. First, the *one-shot* setting where the policy is learned on a fixed population distribution. Second, a *multi-stage* setting where the population evolves over time and both the policy and the outcome model are updated iteratively. In both cases, we rely on Monte Carlo sampling to approximate the expected utility and optimize the policy using gradient-based methods.

B.1 One-shot setting

In the one-shot setting, the population distribution $P(\mathbf{x})$ is assumed to be fixed and known through samples. The goal is to learn a policy that maximizes expected utility on this static population. The Algorithm 1 starts from an initial parameter vector (θ_0), the algorithm iteratively updates the policy by sampling batches of applicants from the population distribution and generating admission decisions according to the current policy. For each sampled batch, multiple action vectors are drawn to approximate the expectation over ($\pi_\theta(\mathbf{a} | \mathbf{x})$). The corresponding utilities are then used to compute a Monte Carlo estimate of the policy gradient, following the formulation in equation 8. Gradient estimates are averaged across samples to reduce variance, and policy parameters are updated using gradient ascent (or equivalently, gradient descent on the loss). The process is repeated until convergence.

Algorithm 1 Policy Gradient for single-shot setting

```

1: Input: population distribution  $P(\mathbf{x})$ , model  $P(\mathbf{y}|\mathbf{x}, \mathbf{a})$ , utility function  $u$ , sample size  $M$ , batch size  $B$ , learning
   rate  $\eta$ , convergence threshold  $\delta$ 
2: Initialize: Policy parameters  $\theta^0$ ,
3: while  $\|\theta^t - \theta^{t-1}\| > \delta$  do
4:   Sample a batch of applicants  $\mathbf{x}_B \sim P(\mathbf{x})$ 
5:   Initialize gradient accumulator  $G \leftarrow 0$ 
6:   for  $k = 1$  to  $M$  do ▷ Monte Carlo Sampling Loop
7:     Evaluate  $\nabla_{\theta_1} U(\pi_{\theta_1}, \mathbf{x})$  or  $\nabla_{\theta_2} \mathcal{L}(\theta_2)$  from  $u, \mathbf{x}$  and  $P$ 
8:     Accumulate gradient:  $G \leftarrow G + \frac{1}{M}(\nabla_{\theta_1} U(\pi_{\theta_1}, \mathbf{x})$  or  $G \leftarrow G + \frac{1}{M}(\nabla_{\theta_2} \mathcal{L}(\theta_2))$ 
9:   end for
10:  Update Policy Parameters:  $\theta_1^{t+1} \leftarrow \theta_1^t + \eta G$  or  $\theta_2^{t+1} \leftarrow \theta_2^t - \eta G$ 
11: end while
12: return  $\pi_{\theta^*}$ 

```

B.2 Multi-stage setting

In the multi-stage setting, the decision process unfolds over multiple stages, where the applicant population evolves over time. At each stage, a new cohort of applicants is observed, decisions are made using current policy, and the results are only partially revealed (for selected individuals). The outcome model and policy are then updated based on the accumulated data.

To capture the evolving population, we use a generative model to simulate new applicants at each stage. The policy is optimized iteratively using policy gradient updates, while the outcome model is refined using newly observed data. This setting reflects realistic scenarios, such as university admissions, where decisions influence future data and must adapt over time.

Algorithm 2 Multi-Stage Policy Gradient with Dynamic Updates

```

1: Input: number of stages  $K$ , base learning rate  $\eta$ , decay factor  $\gamma$ , sample size  $M$ , batch size  $B$ 
2: Initialize: Policy  $\theta^0$ , simulator  $(P^*(\mathbf{x}), P^*(\mathbf{y} | \mathbf{x}))$ , distribution model  $P_t(\mathbf{x})$ , Bayesian outcome predictor  $P_t(\mathbf{y} | \mathbf{x}, \mathbf{a})$ , Applicant History  $\mathcal{H} \leftarrow \emptyset$ , Training Data  $\mathcal{T} \leftarrow \emptyset$ 
3: for  $t = 0$  to  $K - 1$  do
4:    $\eta_t \leftarrow \eta / (1 + \gamma \cdot t)$  ▷ Apply learning rate decay
5:   Step 1: Environment Simulation
6:   Sample synthetic applicant pool  $\mathbf{x}_t \sim P^*(\mathbf{x})$ 
7:   Predict latent outcomes  $\mathbf{y}_t \sim P^*(\mathbf{y} | \mathbf{x})$ 
8:   if  $t == 0$  then
9:      $\mathbf{a}_t \leftarrow$  Apply threshold-based heuristic  $\pi_0(\mathbf{x}_t)$ 
10:  else
11:    Step 2: Policy Optimization
12:    Sample posterior parameters  $\theta_{TS}$  from  $P_t(\mathbf{y} | \mathbf{x}, \mathbf{a})$  ▷ Thompson Sampling
13:    Run Algorithm 1 using  $\theta_{TS}$ 
14:     $\mathbf{a}_t \leftarrow$  Apply learned policy  $\pi_{t-1}(\mathbf{x}_t)$ 
15:  end if
16:  Step 3: Data Collection
17:  Observe outcomes  $\mathbf{y}_{obs}$  for admitted individuals
18:   $\mathcal{D}_{t+1} \leftarrow \mathcal{D}_t \cup \{\mathbf{x}_{admit}, \mathbf{y}_{obs}\}$ 
19:   $\mathcal{H}_{app} \leftarrow \mathcal{H}_{app} \cup \mathbf{x}_t$  ▷ Update applicant history
20:  Step 4: Model Refinement
21:  Update applicant model  $P_t(\mathbf{x})$  on  $\mathcal{H}_{app}$ 
22:  Update Bayesian predictor  $P_t(\mathbf{y} | \mathbf{x}, \mathbf{a})$  with new data  $\mathcal{D}_{k+1}$ 
23: end for
24: return Final optimized policy  $\pi_{\theta^*}$ 

```

C Simple Baseline comparison for the one-shot setting

To evaluate the effectiveness of our policy utility in the one-shot setting, we compare it against two baseline selection methods: Threshold-based Selection and Greedy Selection. These baselines provide useful benchmarks to assess how well our policy approaches optimize admissions decisions in figure 7.

The Threshold-based Selection. This approach admits candidates whose predicted GPA or performance in three key courses exceeds a predefined threshold. It ensures that only applicants meeting a minimum academic standard are selected, offering a straightforward but rigid method that does not consider the overall distribution of applicants' abilities. Although it maintains academic quality, it can exclude candidates who could perform well despite falling just below the threshold.

The Greedy Approach (Based on Total Outcome). It prioritizes the applicants with the highest predicted academic performance by selecting those with the highest sum of grades across all courses. Unlike the threshold method,

which applies a fixed cutoff point, the greedy approach ranks candidates and selects the top performers, assuming that maximizing total academic achievement will yield the best outcomes. However, this method may favor candidates with strong overall scores while neglecting other factors that contribute to long-term success, such as diversity in skills or adaptability to different learning environments. In both methods, the number of selected applicants remains the same.

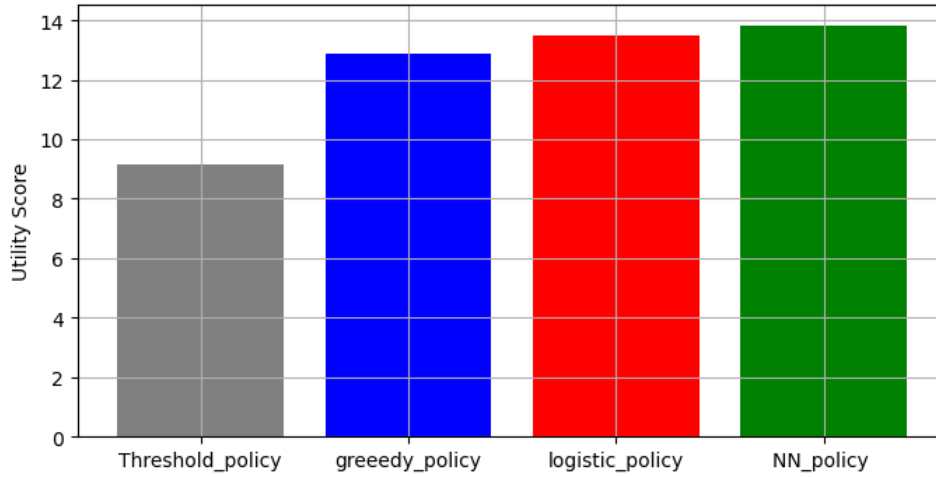


Fig. 7. Comparison of utility scores for different selection strategies: threshold-based selection, greedy selection, and Logistic and neural network-based selection.

The Neural network approach achieves the highest utility, showing that it optimizes student selection more effectively than all other methods. The greedy approach, which selects individuals based on total predicted outcomes, performs better than the threshold method, suggesting that a naive optimization of total scores could yield the best long-term outcomes. The logistic policy can also be seen as a baseline because it has been used and implemented in [22] for the single-shot setting.

D Effect of Highly Delayed Policy Updates for neural network

We further examine the behavior of the neural policy under more extreme update delays, where the policy is updated only every 3, 5, or 6 stages. Figure 8 shows that while the policy can still achieve high utility when updated every 3 years, performance quickly deteriorates when update periods are very late. In particular, we observe that when updates are delayed for too long, the neural policy can start to degrade after several stages (from stage 5), suggesting that although neural policies are robust to moderate delays, excessively infrequent updates lead to unstable performance and poor generalization over time.

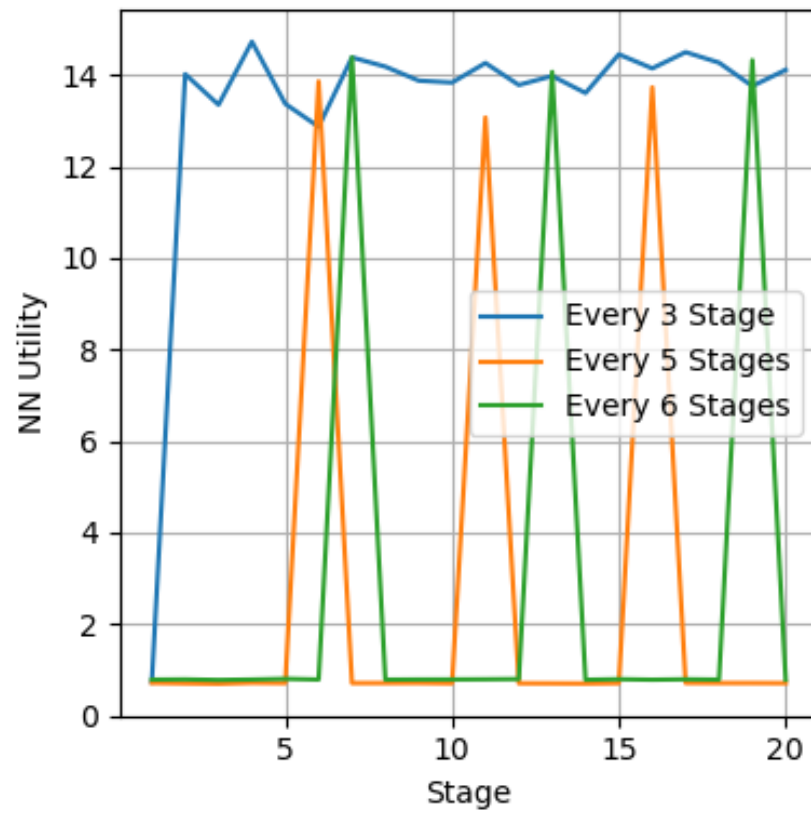


Fig. 8. Average utility across stages under different policy update frequencies under 50 iterations and $c=0.001$.